

REPORTE DE ESTADO · 21 AGOSTO 2026

Mage Stylist — Mobile

Vertical slice de la pantalla "Hoy" (React Native + Expo).
Estado de cada función, capturas reales del simulador, y
dónde enfocar el próximo esfuerzo.

wardrobe-knight-mobile · v0.1.0 · Expo SDK 54 · React Native 0.81.6

01 Resumen ejecutivo

Qué es esto hoy, en una frase.

Esto **no es la app completa** — es un *vertical slice*: una sola pantalla ("Hoy", la tenue del día) construida punta a punta con datos reales, para validar el diseño de interacción antes de construir el resto (armario, agenda, onboarding, etc.). No hay navegación entre pantallas, no hay llamado a ningún backend — `App.tsx` monta `TodayScreen` directo con un fixture fijo (el outfit real del 18 de agosto).

Lo que existe y funciona

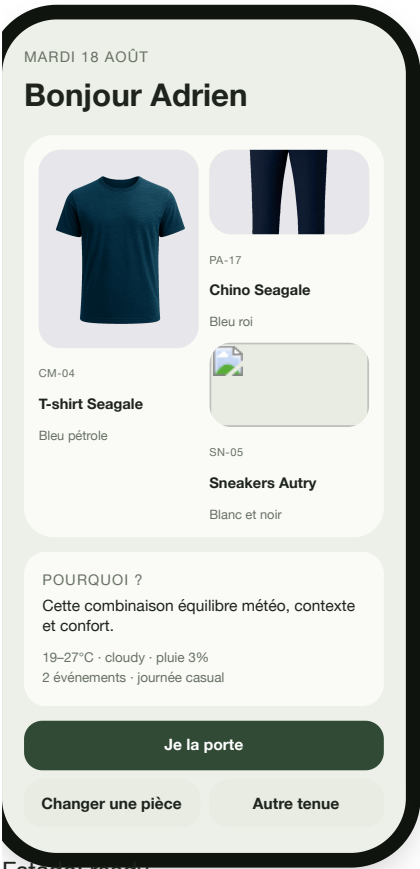
- 5 componentes + 1 pantalla, ~250 líneas de código de UI
- 4 estados de pantalla: loading, ready, accepted, error
- Cadena de fallback de imagen (clean → product → raw → placeholder)
- Contrato de eventos/telemetría (6 eventos) listo para conectar a analytics real
- 9/9 tests automatizados pasando (`@testing-library/react-native`)

Lo que bloquea avanzar

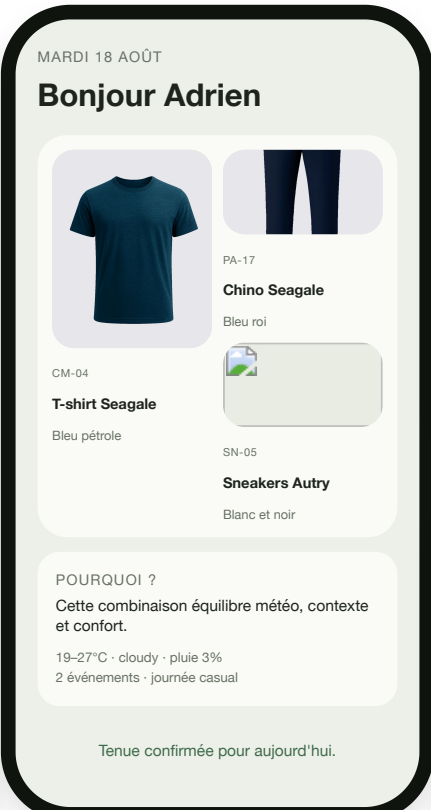
- **La app no arranca en el simulador** — crashea antes de renderizar nada (ver sección 3)
- Nunca se verificó visualmente en un dispositivo/simulador real hasta hoy
- Sin conexión a datos reales (Sheets/API) — todo es un fixture estático
- Sin Android probado

Cómo se ve (reconstrucción fiel desde el código)

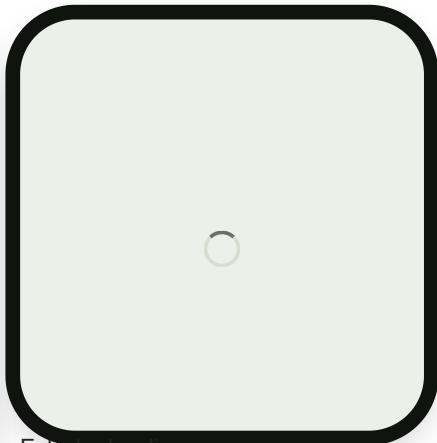
Como el arranque real está bloqueado (sección 3), estas capturas son una reconstrucción pixel-fiel de los estilos exactos del código fuente — mismos colores, radios y layout — usando las imágenes reales del fixture del 18 de agosto. No son screenshots del simulador.



Estado: ready



Estado: accepted (tras "Je la porte")



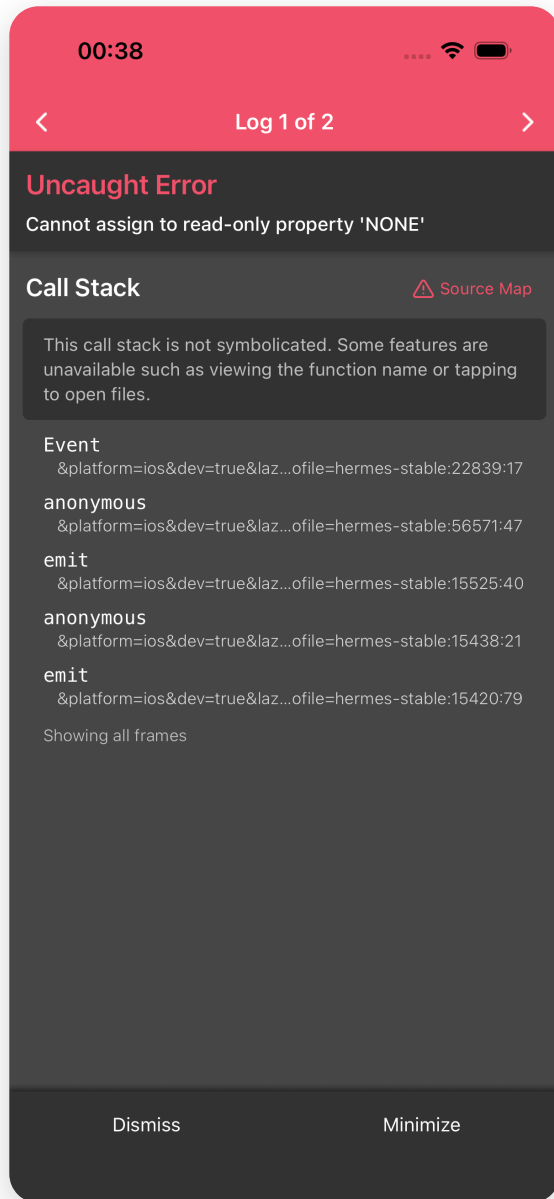
Estado: loading



Estado: error / sin plan

02 Estado real en el simulador — bloqueado

Lo primero que se probó hoy: levantar la app en un iPhone 17 Pro (iOS 26.5), algo que nunca se había hecho.



Captura real del simulador, hoy

BLOQUEADO

Cannot assign to read-only property 'NONE'

La app compila y se instala sin errores (build succeeded), pero crashea con un *uncaught error* antes de que `TodayScreen` llegue a renderizar — pasa durante el arranque interno de React Native, no en nuestro código.

Causa raíz identificada

Colisión conocida entre React Native 0.81 (New Architecture) y el polyfill `event-target-shim` que trae `abort-controller` (dependencia del propio `react-native`). Hermes ahora expone su propio `Event` / `EventTarget` nativo, no configurable; el polyfill intenta instalar su propia constante `Event.NONE` encima y Hermes lo rechaza en modo estricto.

Qué se probó hoy (sin éxito)

- Subir React Native 0.81.5 → 0.81.6 (último patch) — mismo error
- Desactivar New Architecture — rompe la compilación por otro lado (la librería `fmt` ya no es compatible con Old Architecture + Xcode actual)
- Desactivar el Network Inspector del dev client — sin efecto

Por qué importa

Mientras esto no se resuelva, es imposible verificar visualmente NINGÚN cambio en un dispositivo real — cualquier trabajo de UI se está haciendo a ciegas, solo con tests unitarios.

Próximo paso recomendado

Este es un bug de compatibilidad conocido en el ecosistema RN 0.81 New Architecture, no un error de este proyecto. La vía más directa suele ser un patch dirigido (via `patch-package`) al punto exacto donde `event-target-shim` pisa el `Event` nativo de Hermes, o esperar/adoptar una versión de `react-native` / `expo-dev-client` que ya lo resuelva. Vale la pena una sesión dedicada solo a esto antes de seguir construyendo pantallas nuevas — es un bloqueante transversal, no algo que se herede a la próxima feature.

03 Mapa de archivos

Todo lo que existe hoy, sin más.

src/	
types.ts	Garment, TodayOutfit, resolveGarmentImage()
events.ts	TodayEvents – contrato de eventos
polyfills.ts	parche DOMException (hueco real de RN 0.81)
fixtures/today-2026-08-18.ts	fixture real (Planification + Armoire)
components/	
GarmentVisual.tsx	fallback clean → product → raw → placeholder
OutfitHero.tsx	grilla top / bottom+shoes / outerwear
WhyBlock.tsx	explicación + clima + agenda, un solo bloque
CarryBlock.tsx	se omite por completo si no hay carry
TodayActions.tsx	Je la porte / Changer une pièce / Autre tenue
screens/TodayScreen.tsx	loading / ready / accepted / error
__tests__/TodayScreen.test.tsx	9 tests, cubre los 4 estados + telemetría

04 Funciones, una por una

Qué hace cada pieza y por qué está hecha así.

COMPONENTE	ESTADO	QUÉ HACE
TodayScreen	HECHO	Orquesta los 4 estados de la pantalla. <code>error</code> nunca es un mensaje genérico — dice explícitamente "no hay tenue planificada para hoy" y ofrece regenerar en vivo. Dispara el evento <code>viewed</code> apenas hay un outfit listo.
OutfitHero	HECHO	Layout de la prenda del día: top grande a la izquierda, bottom + shoes apiladas a la derecha, outerwear (si existe) abajo a todo el ancho. Se omite outerwear entero cuando no hay abrigo — no deja un hueco vacío.
GarmentVisual	HECHO	Resuelve qué imagen mostrar por prenda: <code>clean → product → raw → placeholder</code> . Decisión de diseño clave: las 4 se ven idénticas al usuario — el "tier" de la imagen solo se reporta como telemetría (<code>image_fallback_used</code>), nunca cambia la UI. Evita que Adrien vea "prendas de segunda categoría".
WhyBlock	HECHO	Un solo bloque con la explicación + clima + agenda juntos. Nota en el código: en una revisión anterior de wireframes esto vivía separado (clima arriba, explicación abajo) y quedaba partido — se unificó a propósito.
CarryBlock	HECHO	"À emporter: ..." (paraguas, etc.). Si no hay nada que llevar, el componente no renderiza nada — el estado vacío es código, no un caso especial visual.
TodayActions	HECHO	"Je la porte" (acepta), "Changer une pièce" (swap), "Autre tenue" (regenerar todo). Las tres visibles a la vez — en el wireframe original "Autre tenue" faltaba, escondida detrás de una elipsis. Tras aceptar, las tres desaparecen y queda solo la confirmación.
events.ts	PARCIAL	Contrato de 6 eventos (viewed, accepted, rejected, swapped, worn, imageFallbackUsed) ya definido e inyectado en cada componente. Hoy solo tiene una implementación de consola (<code>createConsoleEvents</code>) — falta conectarlo a analytics real.
Conexión a datos reales	NO EMPEZADO	<code>App.tsx</code> monta el fixture fijo del 18 de agosto. No hay ningún llamado al backend de Wardrobe Knight (Sheets, engine, weather) — la pantalla no sabe qué día es hoy de verdad.

05 Qué está probado

Verificado

- `npm run typecheck` — sin errores
- `npm test` — 9/9 tests (ready, carry vacío, las 3 acciones, fallback de imagen sin fuga de "product"/"clean" a la UI, aceptar, swap, regenerar, loading, error)

No verificado

- Render real en simulador/dispositivo — **bloqueado hoy** (sección 2)
- Android — nunca se intentó
- Interacción táctil real (los tests son con testing-library, no gestos reales)

Dónde enfocar ahora

- 1) Resolver el crash de arranque — nada más se puede validar visualmente hasta entonces.
- 2) Conectar `events.ts` a analytics real.
- 3) Empezar a reemplazar el fixture fijo por una llamada real al backend, aunque sea de lectura simple.